台灣科技大學 (115) 學年度 Homework 1 Due Date: 115 年 11 月 30 日

科目: **超大型積體電路測試與可測試設計**

Problem 1: Fault Modeling & Collapsing

Consider a 3-input AND gate with inputs *A, B, C,* and output *Z.*

(a) List all the single stuck-at faults for this gate. (3 pts)

(a) Apply fault equivalence and fault dominance reduction techniques to collapse the initial fault set. Show your step-by-step reasoning and state the final minimum collapsed fault Set. (12 pts)

Problem 2: Transistor-Level Fault Models

Consider a 3-input NAND gate with inputs *A, B, C* and output *Z.*

(a) Draw the CMOS circuit diagram for the three-input NAND gate. (3 pts)

(b) Determine the number of transistor faults for both single-fault and multiple-fault assumptions. (6 pts)

(c) How can you test for stuck-open and stuck-on faults in the NAND gate. Provide examples to illustrate your method. (6 pts)

Problem 3: Scan Design

(a) Explain how full-scan design transforms the complexity of Sequential ATPG to combinational ATPG. Why is this transformation a necessity for VLSI testing? (5 pts)

(b) Describe a complete operating sequence for a single scan test pattern. Detail what happens in the circuit during the shift-in phase, capture phase, and shift-out phase. Draw or write out the precise timing behavior of the scan enable (SE) signal relative to the system clock (CLK) across these phases. (15 pts)

Problem 4: Test Time of Scan-Based Design

The major drawback of scan-based *Design-for-Testability (DFT)* is the significant increase in test application time due to the serial shifting mechanism. For a single test pattern, the automatic test equipment (ATE) must serial shift the test stimulus into the scan chain, trigger a functional capture cycle, and serial shift the response out. For a design with *Nv* test patterns, the exact formula for the total number of clock cycles required for a single scan chain containing N_{ff} flip-flops is:

$$Total\ Cycles = (N_{ff} \times N_v) + N_{ff} + N_v$$

$N_{ff} \times Nv$ represents the bits shifted in for N_v patterns.

N_{ff} represents the final shift-out cycle needed to unload the response of the last N_v-*th* pattern. The last N_v represents the capture clock cycles (one capture cycle per pattern). For a multiple scan chain configuration with K parallel scan chains, assuming the flip-flops are balanced evenly, the longest chain length determines the shift time:

$$N_{max_chain} = \lceil N_{ff}/K \rceil$$

The formula updates to:

$$Total\ Cycles = (N_{max_chain} \times N_v) + N_{max_chain} + N_v$$

We assume that an industrial ASIC design contains 60,000 flip-flops (N_{ff} = 60,000). The ATPG tool generates 4,000 test patterns (N_v = 4,000) to achieve the targeted stuck-at fault coverage. The ATE runs the scan shift operations at a frequency of 25 MHz, while the functional capture cycles are executed at 200 MHz.

(a) Assume all 60,000 flip-flops are stitched into a single, long scan chain. Calculate the total number of clock cycles required to execute the entire test suite. (5 pts)

(b) Calculate the actual total test time (in seconds). (Hint: Keep track of which cycles run at the shift frequency vs. the capture frequency). (5 pts)

(c) To reduce the manufacturing test cost, the DFT engineer reconfigures the design into 16 parallel scan chains ($K = 16$), assuming the flip-flops are distributed perfectly evenly among the chains. Determine the length of the longest scan chain (N_{max_chain}). (2 pts)

(d) Calculate the new total test time (in seconds) under this 16-chain balanced configuration. (3 pts)

Problem 5: Test Generation

For the circuit shown in Fig. 1, derive test patterns for detecting x_2 stuck-at 0 and signal 8 stuck at 1. (10 pts)

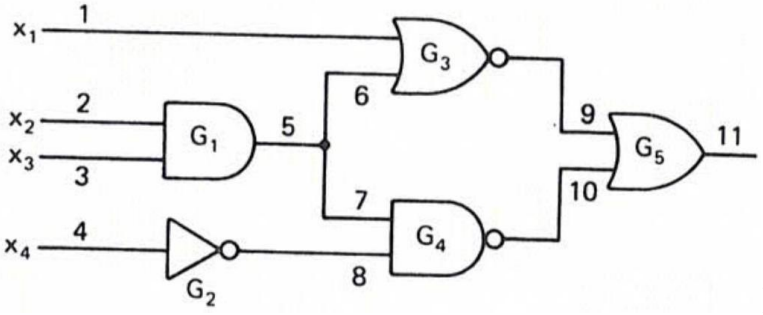


Fig. 1

Problem 6: Parallel Fault Simulation

Assuming a four-bit machine word, demonstrate parallel fault simulation of vector *(x_1, x_2, x_3,x_4) = (0, 1, 0, 1)* for *x_1/1, 5/1*, and *10/0* faults in the shown in Fig. 1 circuit. (15 pts)

Problem 7: Deductive Fault Simulation (15%)

Consider the vectors *(1, 1)* and *(0, 1)* applied to the circuit shown in Fig. 2. We will simulate the *s-a-0* and *s-a-1* faults on all lines *a* through *g* for these vectors using deductive fault simulation. Determine which faults can be detected using these two test patterns. (10 pts)

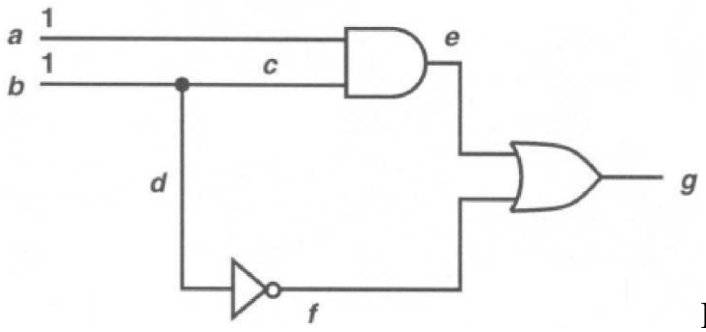


Fig. 2

1.(a)

A SA0

A SA1

B SA0

B SA1

C SA0

C SA1

Z SA0

Z SA1

1.(b)

Step 1：使用 Fault Equivalence（故障等價）化簡

若兩個故障在所有可能的輸入組合下，都會產生相同的錯誤輸出行為，則稱為 Fault Equivalent（故障等價），可以只保留其中一個代表故障。

觀察 SA0 故障

當 \(A\) 發生 SA0 時：

\[Z=0\cdot B\cdot C=0 \]

不論輸入 \(B、C\) 是多少，輸出都固定為 0。

同理：

\(A/0\)：輸出永遠為 0。

\(B/0\)：輸出永遠為 0。

\(C/0\)：輸出永遠為 0。

\(Z/0\)：輸出永遠為 0。

因此，這四個故障彼此等價：

$$\boxed{A/0\equiv B/0\equiv C/0\equiv Z/0}$$

保留一個代表即可，例如 $Z/0$。

原本 8 個故障經過等價化簡後，剩下：

$$F_{\text{eq}}=\{Z/0,A/1,B/1,C/1,Z/1\}$$

故障數由 8 個減少至 5 個。

Step 2：使用 Fault Dominance（故障支配）化簡

故障支配的核心概念是：如果測試某個故障所使用的測試向量，也必定能偵測另一個故障，則後者可以在故障覆蓋的角度被移除。

先找出各個 SA1 故障的測試向量。

對於輸入 $A/1$，必須讓正常電路輸出為 0，而故障電路輸出為 1。

選擇：

$$(A,B,C)=(0,1,1)$$

正常電路：

$$Z=0\cdot1\cdot1=0$$

故障電路（A 被固定為 1）：

$$Z_f=1\cdot1\cdot1=1$$

所以測試向量 011 可以偵測 $A/1$。

同理可得：

故障	測試向量 ABC	正常輸出 Z	故障輸出 Z_f
$A/1$	011	0	1
$B/1$	101	0	1
$C/1$	110	0	1

接著分析輸出故障 $Z/1$。

只要正常輸出 $Z=0$，就能偵測 $Z/1$，因為故障輸出會固定為 1。

因此，上述三個測試向量 \(011、101、110\) 也都能偵測 \(Z/1\)。

也就是說：

\[T(A/1)\subseteq T(Z/1) \]

\[T(B/1)\subseteq T(Z/1) \]

\[T(C/1)\subseteq T(Z/1) \]

其中 \(T(f)\) 代表能偵測故障 \(f\) 的測試向量集合。

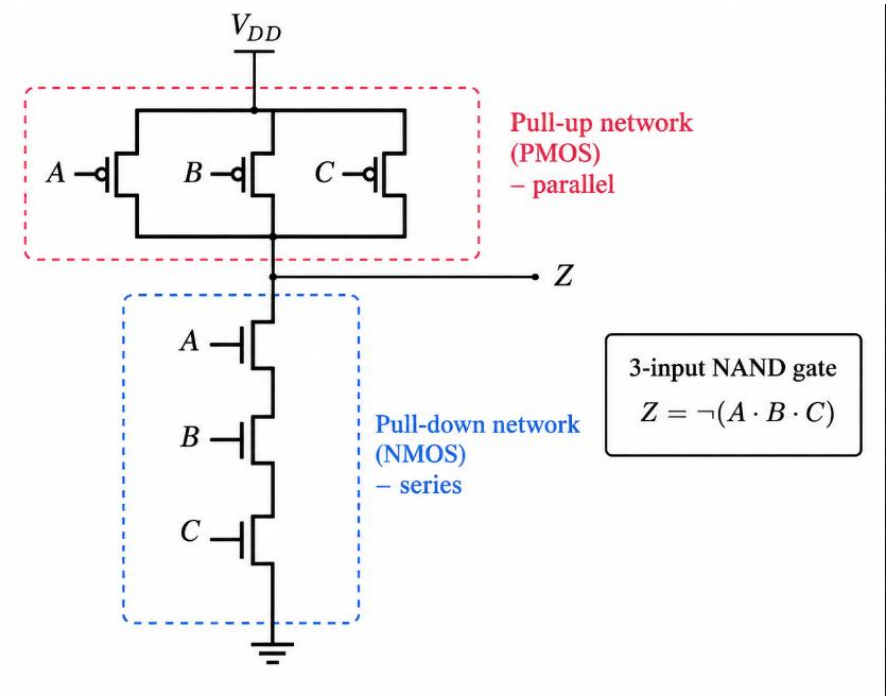
因此，保留 \(A/1、B/1、C/1\) 時，\(Z/1\) 已可由這些故障的測試向量一併偵測，不必再單獨保留 \(Z/1\)。

移除 \(Z/1\) 後，得到最終化簡集合。

Step 3：最終 Minimum Collapsed Fault Set

\[\boxed{F_{\text{collapsed}}=\{Z/0,A/1,B/1,C/1\}} \]

2.(a)



2.(b)

單一故障假設： 6 + 6 = 12

多重故障假設：

每個電晶體在物理上存在 3 種互斥狀態：3^6 - 1 = **728**

1. 正常（Fault-free）
2. Stuck-Open
3. Stuck-On

2.(c)

Stuck-open fault：

由於 CMOS 電路可能保留先前的輸出電荷，因此需要利用兩個連續的測試向量來偵測。

當輸入由 \(000\) 變成 \(111\) 時，正常電路的輸出應由 1 變為 0。若串聯 NMOS 中有一顆 stuck-open，放電路徑會中斷，輸出可能保持在 1。

Stuck-on fault：

當輸入$(A, B, C) = (0,1,1)$，正常情況下，A 控制的 NMOS 應截止，但如果該 NMOS 持續導通，另外兩顆 NMOS 又因 B、C 為 1 而導通，便可能形成從輸出到 GND 的放電路徑，使輸出由預期的 1 變成 0，從而偵測故障。

3.(a)

將所有FF改為scan FF並將其串為scan chain，scan chain讓 ATPG 可以直接透過scan in觀察scan out並

得到內部狀態訊息。

降低 ATPG 複雜度、提高可控制性、提高可觀察姓、提高故障覆蓋率

3.(b)

1. Shift-in phase（移入測試資料）

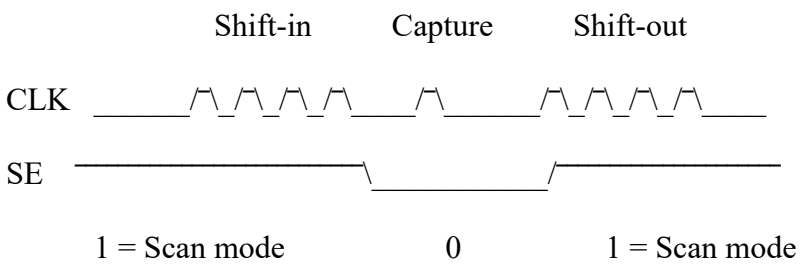
- 將 SE 設為 1，啟用 Scan mode。
- 每個有效的 CLK 時脈邊緣，測試資料透過 Scan-In（SI）逐位移入 Scan Chain。
- 原本儲存在 Scan FF 中的資料會逐級移動，直到完整的測試向量載入所有 Scan FF。
- 此階段主要用於設定內部電路狀態，不是執行正常功能運算。

2. Capture phase（擷取測試反應）

- 在 Shift-in 完成後，將 SE 設為 0，切換至 Normal mode。
- 施加一個或多個有效的 CLK 時脈邊緣，讓組合邏輯根據載入的測試狀態與外部輸入進行運算。
- Scan FF 擷取組合邏輯的輸出，將測試反應儲存起來。

3. Shift-out phase（移出測試反應）

- 將 SE 設回 1，重新啟用 Scan mode。
- 每個有效的 CLK 時脈邊緣，將 Scan FF 內儲存的測試反應逐位移出至 Scan-Out（SO）。
- 同時，新的資料也可能從 SI 移入。
- 將讀出的結果與預期的正確結果（Expected Response）比較，即可判斷是否存在故障。



4.(a)

$$\begin{aligned}\text{Total Cycles} &= (60{,}000 \times 4{,}000) \\ &\quad +60{,}000 + 4{,}000 \\ &= 240{,}000{,}000 + 60{,}000 + 4{,}000 \\ &= \boxed{240{,}064{,}000}\end{aligned}$$

4.(b)

計算 Shift 時間

$$\begin{aligned}C_s &= N_{ff}N_v + N_{ff} \\ &= (60{,}000 \times 4{,}000) + 60{,}000 \\ &= 240{,}060{,}000\end{aligned}$$

Shift 頻率為 25 MHz，即每秒 25,000,000 個週期。

$$\begin{aligned}T_s &= \frac{C_s}{f_s} \\ &= \frac{240{,}060{,}000}{25 \times 10^6} \\ &= \boxed{9.6024\text{秒}}\end{aligned}$$

計算 Capture 時間

$$C_c = N_v = 4{,}000$$

Capture 頻率為 200 MHz。

$$\begin{aligned}T_c &= \frac{C_c}{f_c} \\ &= \frac{4{,}000}{200 \times 10^6} \\ &= \boxed{0.00002\ \text{秒}}\end{aligned}$$

計算總測試時間

$$\begin{aligned}T_{\text{total}} &= T_s + T_c \\ &= 9.6024 + 0.00002 \\ &= \boxed{9.60242\ \text{秒}}\end{aligned}$$

4.(c)

$$N_{\text{max_chain}} = \left\lceil \frac{N_{ff}}{K} \right\rceil$$

$$N_{\text{max_chain}} = \left\lceil \frac{60{,}000}{16} \right\rceil = \lceil 3{,}750 \rceil$$

4.(d)

$$\boldsymbol{T_{\text{shift}} = \frac{N_v N_{\text{max_chain}} + N_{\text{max_chain}}}{f_s}}$$

$$\boldsymbol{= \frac{4{,}000(3{,}750) + 3{,}750}{25 \times 10^6} = 0.60015\text{s}}$$

Capture 時間不變：

$$T_{\text{capture}} = \frac{N_v}{f_c} = \frac{4{,}000}{200 \times 10^6} = 0.00002 \text{ s}$$

$$\begin{aligned} T_{\text{total}} &= T_{\text{shift}} + T_{\text{capture}} \\ &= 0.60015 + 0.00002 \\ &= \boxed{0.60017\text{seconds}} \end{aligned}$$

5.

x_2 stuck-at 0

1. 故障激發（Fault Activation）

欲激發信號 2 (x2) 的 s-a-0 故障，必須將正常邏輯值設為 1：

- 設定 **x2 = 1**（此時信號 2 呈現 **D** 狀態，即 1/0）。

2. 故障傳播至扇出點（Propagation to Node 5）

欲使信號 2 的 **D** 通過 AND 閘 G1 傳播至信號 5，G1 的另一輸入端必須設為非控制值（Non-controlling value）1：

- 設定 **x3 = 1**（信號 3 = 1）
- 此時信號 5 = **D**，扇出分支信號 6 = **D**、信號 7 = **D**。

3. 選擇並敏化傳播路徑至主要輸出（信號 11）

信號 5 分支為兩條路徑（經由 G3 或 G4），需分析哪一條路徑可將故障效應成功傳播至輸出端 11：

- **嘗試路徑 A（經由 G3 -> 信號 9 -> G5）：**
  - 欲使信號 6 (D) 通過 NOR 閘 G3，需設信號 1 (**x1**) = 0，此時信號 9 = **D'**（即 0/1）。
  - 接著欲使信號 9 (D') 通過 OR 閘 G5，另一輸入信號 10 必須為 OR 閘的非控制值 0。
  - 然而，信號 10 是 NAND 閘 G4 的輸出。欲使信號 10 恆為 0，G4 的兩個輸入（信號 7 與信號 8）必須同時為 1。但目前信號 7 為 **D**（故障時為 0），在故障發生時信號 10 必定變為 1，這會遮蔽（Mask）信號 9 的故障效應。因此**無法單獨透過路徑 G3 傳播**。
- **選擇路徑 B（經由 G4 -> 信號 10 -> G5）：**

  1. **通過 NAND 閘 G4：** 欲使信號 7 (D) 通過 G4，另一輸入端信號 8 必須設為 NAND 閘的非控制值 1。
     - 設定 **信號 8 = 1** → 反推經由反相器 G2，設定 **x4 = 0**。
     - 此時信號 10 = (D NAND 1) = **D'**（即 0/1）。
  2. **通過 OR 閘 G5：** 欲使信號 10 (D') 通過 OR 閘 G5 到達輸出 11，另一輸入端信號 9 必須穩定為 OR 閘的非控制值 0（避免被同源扇出信號干擾）。
     - 信號 9 為 NOR 閘 G3 的輸出。只要將 G3 的輸入端信號 1 (**x1**) 設為 NOR 閘的控制值 1，不論信號 6 為何，信號 9 都會強制鎖定為 0。
     - 設定 **x1 = 1 → 信號 9 = 0**。
     - 此時主要輸出 **信號 11 = (0 OR D') = D'**（正常輸出為 0，故障輸出為 1），故障成功傳播至輸出！

- *(註：若設 x1 = 0, x4 = 0，則信號 9 = D′ 且信號 10 = D′，經過 OR 閘 G5 後，D′ OR D′ = D′，多重路徑敏化同樣成立！)*

x2 s-a-0 測試向量結果

- **(x1, x2, x3, x4) = (1, 1, 1, 0)** 或 **(0, 1, 1, 0)**

signal 8 stuck-at-1

1. 故障激發（Fault Activation）与前向反推（Justification）

欲激發信號 8 的 s-a-1 故障，必須將信號 8 的正常邏輯值設為 0（使信號 8 呈現 **D'**，即 0/1）：

- 欲使信號 8 = 0，經由反相器 G2 反推，必須設定輸入 **x4 = 1**（信號 4 = 1）。

2. 故障傳播（Fault Propagation）

信號 8 僅有一條通往輸出端 11 的路徑（信號 8 -> G4 -> 信號 10 -> G5 -> 信號 11）：

1. **通過 NAND 閘 G4（傳播至信號 10）：**
   - 欲使信號 8 (D') 通過 NAND 閘 G4，另一輸入端 **信號 7 必須設為非控制值 1**。
   - 由於信號 7 來自信號 5（AND 閘 G1 的輸出），欲使信號 5 = 1，必須將 G1 的兩個輸入端皆設為 1：
     - 設定 **x2 = 1** 且 **x3 = 1**。
   - 此時信號 7 = 1，經過 G4 後，**信號 10 = (1 NAND D') = D**（即 1/0）。
2. **通過 OR 閘 G5（傳播至信號 11）：**
   - 欲使信號 10 (D) 通過 OR 閘 G5，另一輸入端 **信號 9 必須為非控制值 0**。
   - 檢查信號 9 的狀態：在前一步驟中，因為 x2 = 1 且 x3 = 1，使得信號 5 = 1，進而使分支 **信號 6 = 1**。
   - 由於信號 6 = 1（已為 NOR 閘 G3 的控制值），不論信號 1 (x1) 為 0 或 1，NOR 閘 G3 的輸出 **信號 9 必定恆為 0**：
     - **x1 = X**（Don't care，可為 0 或 1）。
   - 此時主要輸出 **信號 11 = (0 OR D) = D**（正常輸出為 1，故障輸出為 0），故障成功傳播至輸出！

3. signal 8 s-a-1 測試向量結果

- **(x1, x2, x3, x4) = (X, 1, 1, 1)**，即 **(0, 1, 1, 1)** 或 **(1, 1, 1, 1)**

6.

建立 4-bit 平行模擬字

指定 bit lane 順序如下，從 MSB 到 LSB：

Bit　代表的電路

\(b_3\) Good circuit（無故障）

\(b_2\) \(x_1/1\) fault

\(b_1\) \(5/1\) fault

\(b_0\) \(10/0\) fault

因此，每條訊號線的 4-bit word 都同時包含四種情況。

3. 逐節點計算

Step 1：輸入端

依據輸入向量 \((0,1,0,1)\)，並只在指定故障所在的 lane 注入故障：

節點　4-bit word 說明

\(x_1\) 0100　　\(x_1/1\) fault lane 強制為 1

\(x_2\) 1111　　所有 lane 均為 1

\(x_3\) 0000　　所有 lane 均為 0

\(x_4\) 1111　　所有 lane 均為 1

Step 2：計算節點 5（\(G_1\)）

\(G_1\) 是 AND gate，正常輸出為

\[5=x_2\land x_3 \]

\[1111\land0000=0000 \]

再將 \(5/1\) fault 注入 \(b_1\)，強制該 bit 為 1：

\[\boxed{5=0010} \]

Step 3：計算節點 8（\(G_2\)）

\(G_2\) 是 NOT gate：

\[8=\overline{x_4} =\overline{1111} \]

\[\boxed{8=0000} \]

Step 4：計算節點 9（\(G_3\)）

\(G_3\) 是 NOR gate：

\[9=\overline{x_1\lor5} \]

先計算 OR：

\[0100\lor0010=0110 \]

再取反：

\[\boxed{9=1001} \]

Step 5：計算節點 10（\(G_4\)）

\(G_4\) 是 NAND gate：

\[10=\overline{5\land8} \]

\[\overline{0010\land0000} =\overline{0000} =1111 \]

最後將 \(10/0\) fault 注入 \(b_0\)，強制該 bit 為 0：

\[\boxed{10=1110} \]

Step 6：計算輸出節點 11（\(G_5\)）

\(G_5\) 是 OR gate：

\[11=9\lor10 \]

\[1001\lor1110 =\boxed{1111} \]

4. 最終平行故障模擬結果

節點	邏輯運算	4-bit 結果
\(x_1\) 輸入		0100
\(x_2\) 輸入		1111
\(x_3\) 輸入		0000
\(x_4\) 輸入		1111
5	\(x_2\land x_3\)，注入 \(5/1\)	0010
8	\(\overline{x_4}\)	0000
9	\(\overline{x_1\lor5}\)	1001
10	\(\overline{5\land8}\)，注入 \(10/0\)	1110
11	\(9\lor10\)	1111

將輸出 word 依序解讀：

電路	輸出 11	故障是否被偵測？
Good circuit	1	—
\(x_1/1\)	1	否
\(5/1\)	1	否
\(10/0\)	1	否

結論： 輸出 word 為 1111，四個 lane 的輸出皆為 1。因此，對於指定輸入向量 \((0,1,0,1)\)，三個故障 \(x_1/1\)、\(5/1\) 與 \(10/0\) 都沒有在主要輸出節點 11 上被偵測到。

7.

根據圖中的電路：

- \(c=b\)，\(d=b\)：都是輸入 \(b\) 的分支。
- \(e=a\land c=a\land b\)：AND gate。
- \(f=\overline d=\overline b\)：NOT gate。
- \(g=e\lor f\)：OR gate。

$$\boxed{g = (a \wedge b) \vee \overline{b}}$$

測試向量一：\((a,b)=(1,1)\)

$$g = 1 \vee 0 = \boxed{1}$$

Fault	故障影響	故障輸出 \(g\)	偵測？
\(a/0\)	\(e=0\)	0	是
\(a/1\)	原本 \(a=1\)	1	否
\(b/0\)	\(c=d=0,\ f=1\)	1	否
\(b/1\)	原本 \(b=1\)	1	否
\(c/0\)	\(e=0\)	0	是
\(c/1\)	原本 \(c=1\)	1	否
\(d/0\)	\(f=1\)	1	否
\(d/1\)	原本 \(d=1\)	1	否
\(e/0\)	\(e=0\)	0	是
\(e/1\)	原本 \(e=1\)	1	否
\(f/0\)	原本 \(f=0\)	1	否
\(f/1\)	\(f=1\)	1	否
\(g/0\)	輸出強制為 0	0	是
\(g/1\)	原本 \(g=1\)	1	否

$$\boxed{F_{(1,1)} = \{a/0, c/0, e/0, g/0\}}$$

測試向量二：\((a,b)=(0,1)\)

$$g = 0 \vee 0 = \boxed{0}$$

Fault	故障影響	故障輸出 \(g\)	偵測？
\(a/0\)	原本 \(a=0\)	0	否
\(a/1\)	\(e=1\)	1	是
\(b/0\)	\(c=d=0,\ f=1\)	1	是
\(b/1\)	原本 \(b=1\)	0	否

\(c/0\) \(e=0\) 0 否

\(c/1\) 原本 \(c=1\) 0 否

\(d/0\) \(f=1\) 1 是

\(d/1\) 原本 \(d=1\) 0 否

\(e/0\) 原本 \(e=0\) 0 否

\(e/1\) \(e=1\) 1 是

\(f/0\) 原本 \(f=0\) 0 否

\(f/1\) \(f=1\) 1 是

\(g/0\) 原本 \(g=0\) 0 否

\(g/1\) 輸出強制為 1 1 是

因此，第二組向量能偵測的故障集合為：

$$\boxed{F_{(0,1)} = \{a/1, b/0, d/0, e/1, f/1, g/1\}}$$

最終答案：兩組測試向量的故障覆蓋率

$$F_{\text{detected}} = \{a/0, a/1, b/0, c/0, \\ d/0, e/0, e/1, f/1, g/0, g/1\}$$